Live FPGA-Accelerated Speech-to-Text on the Kria KV260

I. Big picture and motivation

More than 1.5 billion people live with some level of hearing loss, and 430 million of them have disabling hearing loss [1]. Deaf and hard-of-hearing people rely on captions to follow speech, and a caption only helps while the speaker is still talking. We will build a live captioning device on the AMD Kria KV260, with all processing on the board so the audio never leaves it. One second is the limit for a user's flow of thought [2], and caption users already rate a 2 s delay lower [3], so captions must appear within 1 s.

Speech-to-text is dominated by multiply-accumulate (MAC) operations. The four Cortex-A53 cores run them a few at a time, while the programmable logic (PL) has 1,248 DSP slices that can run hundreds per clock [4]. We will move this arithmetic into the PL, keep the processing system (PS) for control and sequential work, and measure whether the hybrid beats the CPU alone.

The specification's Voice Recognition idea says full speech-to-text is unlikely to run locally. Our project tests that claim on real hardware. We will measure latency, accuracy and power to judge how practical live captioning is on an embedded board. Speed-up and latency are our main goals, and we optimise power only if time allows.

Objective. We will deliver a KV260 that captions speech from a MEMS microphone with a 95th-percentile latency of at most 1000 ms. It will run QuartzNet with its convolutions on a PL INT8 engine that is at least 2× faster than the same model in INT8 on the CPU, at a word error rate (WER) of at most 10 %.

II. Model selection

How much work the PL can take depends on the model, so we chose the model first.

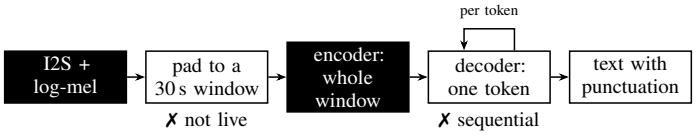
Selection criteria. We judged each candidate on the three tests in Table I, in order.

Comparison. Table II gives our CPU-only benchmark on the KV260. Below each model is its flow from microphone to text; black boxes suit the PL and white boxes stay on the PS.

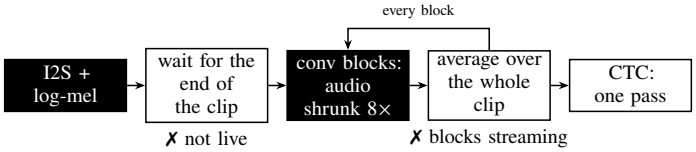
TABLE I
Selection criteria

Criterion	Test	Why it matters
Live	Text from partial audio; no step needs the whole clip or a fixed window	Captions must keep up with the speaker
PL fit	Regular MACs; no token-by-token loop on the critical path	Only parallel work speeds up on the PL
Measured	WER and CPU real-time factor (RTF) on the KV260	The choice rests on our own data

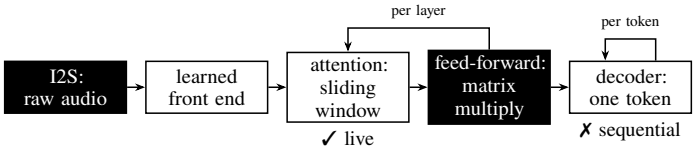
Whisper [5] always encodes a 30 s window, so 3 s of speech costs as much as 30 s, and its decoder writes one token per loop. Even Whisper Base is slower than real time (RTF 1.13).



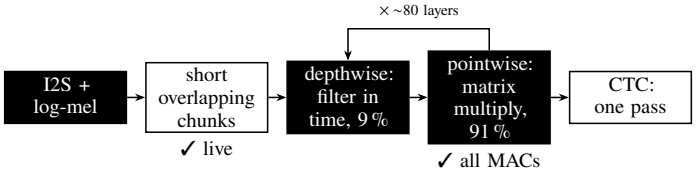
Citrinet [6] shrinks the audio 8× early and is the fastest model we measured (RTF 0.063), but every block averages each channel over the whole clip, so no text appears until the speaker stops.



Moonshine Streaming Tiny [7] reads raw audio and is built for live use, but at RTF 0.998 it has no headroom on the CPU, its WER is 8.32 %, and its decoder is sequential.



QuartzNet 15×5 [8] passes all three tests: about 80 layers of separable convolution, each a depthwise filter in time (9 % of the MACs) and a pointwise matrix multiply (91 %), then a single-pass CTC decode. It reaches 5.43 % WER at RTF 0.198. It was trained on whole utterances, but each layer only sees nearby frames, so we run it live on short, overlapping chunks.



Reach goal: Moonshine. Moonshine Streaming Tiny is our reach goal. It only just keeps up with real time on the CPU, so it gains the most from acceleration, and its encoder projections and feed-forward layers are matrix multiplies for the same engine. In one planned update (Fig. 1) every encoder layer goes to the PL and back, so these round trips and the PS decoder limit the gain. We start it only after milestone M2 (Section V).

III. Technical strategy

Both models reduce to INT8 matrix multiplies, so our strategy is built around one general engine. A block moves to the PL only if it beats the PS cores on the board, so we never add hardware for its own sake.

TABLE II
CPU-ONLY BENCHMARK ON THE KV260

Model	Runtime	p50 (s)	p90 (s)	Max (s)	RTF	WER (%)	RSS (MiB)	Live	PL fit
Citrinet-256	ONNX fp32	0.414	0.835	1.041	0.063	5.55	165	✗	✗
Citrinet-512	ONNX fp32	1.331	2.632	3.133	0.188	5.78	278	✗	✗
QuartzNet 15×5	ONNX fp32	**1.334**	**2.709**	**3.579**	**0.198**	**5.43**	**200**	✓	✓
Citrinet-1024 ($\gamma = 0.25$)	ONNX fp32	3.941	8.804	11.527	0.607	3.24	708	✗	✗
Moonshine Streaming Tiny	ORT quantised	6.722	12.244	18.080	0.998	8.32	313	✓	part
Whisper Base	CTranslate2 INT8	8.498	10.555	11.936	1.125	7.40	371	✗	✗
Moonshine Streaming Small	ORT quantised	15.859	27.191	38.087	2.140	5.55	563	✓	part
Whisper Small	CTranslate2 INT8	27.171	30.848	35.778	3.582	4.97	684	✗	✗
Whisper Large-v3-Turbo	CTranslate2 INT8	119.074	122.460	125.553	15.521	3.12	1675	✗	✗

40 LibriSpeech test clips [9] totalling 307 s, 4 threads, 30 s cool-down. p50, p90 and Max are the median, 90th-percentile and worst processing time per clip. RTF is processing time divided by audio length. WER is the word error rate, so lower is better. RSS is peak resident memory. Live, PL fit: Table I (*part*: the decoder stays on the PS).

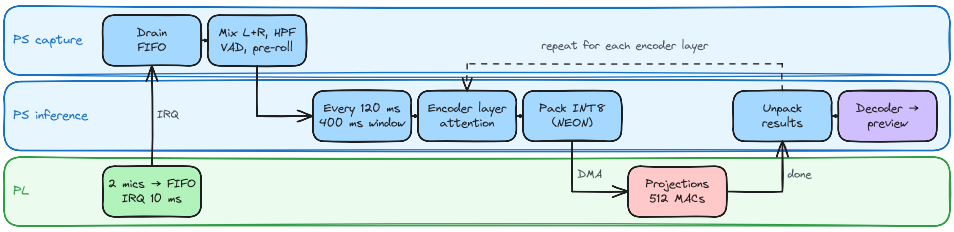


Fig. 1. One planned Moonshine update (reach goal) from our prototype design, with the two-microphone stretch: each encoder layer goes to the engine and back.

TABLE III
AUDIO PATH

Stage	Where	Setting
I2S and FIFO	PL	One SPH0645; 1.024 MHz BCLK, 16 kHz; 1024 samples
FFT and mel	PL, fixed point	20 ms window, 10 ms hop, 512 points, 64 bands
Log, normalise, VAD	PS, float	Running per-band statistics; 700 ms silence ends a phrase
Beamformer	PL, stretch	Delay-and-sum with a second SPH0645

What runs where. In Fig. 2, regular, parallel work goes on the PL: capture, the FFT front end and both kinds of convolution. Control stays on the PS, including the caption states of Fig. 3: a preview every 120 ms while the speaker talks, then, after 700 ms of silence, one pass over the whole phrase, which matches how QuartzNet was trained and repairs errors at chunk edges.

Audio front end. Table III lists the audio path, which matches QuartzNet's training. QuartzNet normalised each band over the whole utterance; live, we use running statistics. Two microphones give at most about 3 dB against uncorrelated noise, so we keep the beamformer only if it lowers WER on a noisy test.

The accelerator. The engine (Fig. 4) is a systolic array of 512 INT8 MACs on DSP48E2 slices, peaking at 51.2 G MAC/s at 100 MHz. QuartzNet needs about 0.96 G MAC per second of audio, so a preview every 120 ms over a 1 s window needs about 8 G MAC/s. Weight tiles stream from DDR into a URAM cache, batch normalisation is folded into the weights, and ReLU and requantisation happen at the array output. Because each value fetched from memory is reused 16 or 32 times, we expect the array to stay busy if DMA delivers the next tile while it computes the current one. Keeping the array fed is the hard part, so we build it in the stages of Table IV.

TABLE IV
GOALS AND ENGINE STAGES (S1 MUST BE BIT-EXACT; EACH LATER STAGE MUST ALSO BE FASTER THAN THE CPU-ONLY BUILD)

Tier	Stage	When	On the PL	Commands
Core	S1	Week 8	Pointwise layers	One per layer; depthwise on the PS
Core	S2	Week 9	Pointwise and depthwise	One per chunk; no round trips between layers
Reach	S3	After M2	Moonshine encoder	One per layer (Fig. 1)
Fallback	–	–	Nothing: QuartzNet INT8 on the CPU	Guarantees a demo

Number format. The engine multiplies 8-bit weights (INT8) and activations (UINT8) into 32-bit sums, then requantises with a bias and a Q31 scale. An INT8 MAC needs one DSP slice instead of two to three for float32, and about 0.3 pJ instead of 4.6 pJ [10]; it still gives about 50 dB of signal-to-quantisation noise, and costs QuartzNet-class models under one WER point [11]. Only the logarithm and normalisation stay in float.

Interfaces and advanced techniques. We commit to the five techniques in Table V. The HP ports are not cache-coherent, so the driver flushes buffers before the PL reads them and invalidates them before the PS reads results; the engine needs no cache controller, as its weights are read-only and its URAM is a scratchpad.

Constraints. Table VI lists the KV260's limits [4]: moving data, not arithmetic, decides the speed-up.

IV. ACCEPTANCE CRITERIA

Goals. Table IV sets three tiers: we commit to the core (S1, S2), start the reach goal (S3) only after M2, and keep

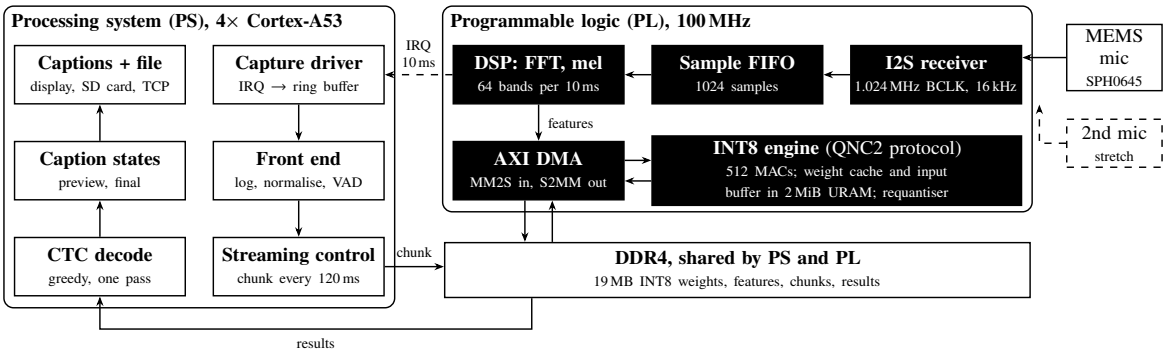


Fig. 2. Planned system. Black: PL; white: PS or off chip. The engine fetches weights and inputs from the shared DDR over AXI DMA.

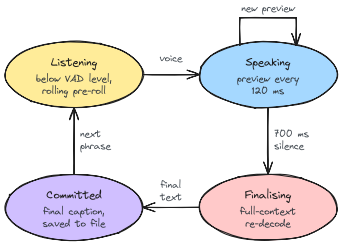


Fig. 3. Caption states, from our prototype design.

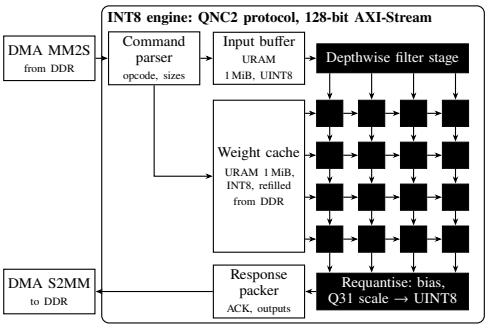


Fig. 4. The INT8 engine (4×4 of the 32×16 systolic array): weights move right, frames move down, each cell keeps a 32-bit sum. Black: arithmetic; white: data movement.

TABLE V
ADVANCED TECHNIQUES

Technique	Where	Purpose
FFT feature extraction	PL	Log-mel front end
Interrupts	PL → PS	Wake every 10 ms instead of polling
DMA	PL ↔ DDR	Features, weights and results
Multi-threading	PS	Capture, inference, display on own cores
Cache coherency	PS ↔ PL	Flush and invalidate shared buffers
Stretch	–	Beamforming, button, LED, SD, TCP

TABLE VI
CONSTRAINTS

Constraint	Value	Consequence
DSP slices	1,248	Bounds the array size
On-chip RAM	~3 MB vs 19 MB of weights	Weights stream from DDR per chunk
HP port	128-bit, ~1.6 GB/s	~12 ms per pass over all weights
DDR	Shared by PS and PL	Transfers compete with the CPU
Clock	100 MHz	Sets the 51.2 G MAC/s peak
Deadline	One update per 120 ms	Each chunk must finish before the next

QuartzNet INT8 on the CPU as the fallback, so the demo never relies on the riskiest work.

Success metrics. Speed-up (Table VII) is measured against the same model in INT8 on the CPU, so the gain comes from the PL, not the number format; this is why it is lower than the 3× against float32 in our first standup. Preview latency is the age of the oldest audio in each update, including queueing. WER follows the Artificial Analysis method [14].

TABLE VII
SUCCESS METRICS

Metric	Target	Stretch	Method
Speed-up vs CPU INT8	≥ 2×	≥ 3×	Paired A/B runs
Preview latency p95	≤ 1000 ms [2], [3]	≤ 250 ms [12], [13]	Paced replay
WER, finals	≤ 10 % [16]	INT8 ≤ float + 1 [11]	VoxPopuli [15], own speech
Board power	≤ 20 W	≤ 15 W [17]	`xmutil`
PS load	Reported	–	`top`
Reach goal	Moonshine live on the engine	–	Same harness

TABLE VIII
INTEGRATION AND SYSTEM TEST TIMELINE

Test, when	Subsystems	Check → pass
U, by M1	Each, by its owner	Unit tests of Table IX → all pass
I1, wk 9	Front end, data, I/O	WAV replay to DDR features → match unit reference
I2, wk 9	I/O, data, accelerator	Saved chunks through the engine → CTC identical to CPU integer reference
I3, wk 9	Accelerator, display	Engine outputs to captions → golden transcript on screen
T1, M2	All (lead Henry)	Live: latency, WER, A/B speed-up → Table VII core targets
T2, wk 10	All	Power, 30 min soak, demo rehearsal → no dropped samples

Test plan. Each owner unit-tests their own subsystem against recorded inputs and a golden INT8 QuartzNet in Python, and every unit test must pass by M1 (Table IX). Integration and system tests then follow the timeline in Table VIII. We merge an optimisation only if the regression suite passes, the PL still matches the integer reference and transcripts are no worse: a faster change that adds word errors is rejected.

Test setup. On the bench (Fig. 5) a PC plays clips from LibriSpeech [9], VoxPopuli [15], text-to-speech and 30 min of our own speech, then scores the timestamped captions; the CPU-INT8 and PL builds alternate on the same clips. A second speaker adds side noise for the beamforming stretch.

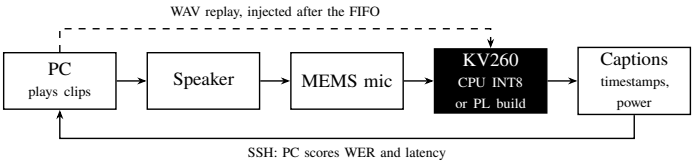


Fig. 5. Test bench; the dashed path skips the room for A/B runs.

TABLE IX
SUBSYSTEMS, OWNERS (B: BACKUP) AND UNIT TESTS

Subsystem, owner	Scope	Unit test → pass
DSP front end Wiki Lauer (b: Pallas)	I2S, FIFO, FFT, mel; button, LED; beamformer (stretch)	Simulation vs Python features; mic self-test → WER unchanged with PL features; clock within 1 %
Accelerator Jason Fang (b: Wiki)	MAC array, depthwise stage, weight cache, requantiser	Every layer vs the INT8 golden model, edge sizes → bit-exact (S1 by M1, S2 by I2); 100 MHz timing met
Data interface Pallas Choi (b: Jason)	AXI DMA chaining, QNC2 parser and packer, double-buffered tiles	DMA loopback of golden vectors and bad packets → bit-exact, recovers; ≥ 1 GB/s
PS–PL I/O Henry Nguyen (b: Kanishk)	Driver, IRQ, cache coherency, threads, chunking, test harness	30 min IRQ run; flush/invalidate pattern test → no missed frames or stale reads
Model, display Kanishk Aggarwal (b: Henry)	Int8 export, golden model, CTC, caption states, DisplayPort text, SD	Golden model vs float; recorded outputs to screen → within 1 WER point; correct captions

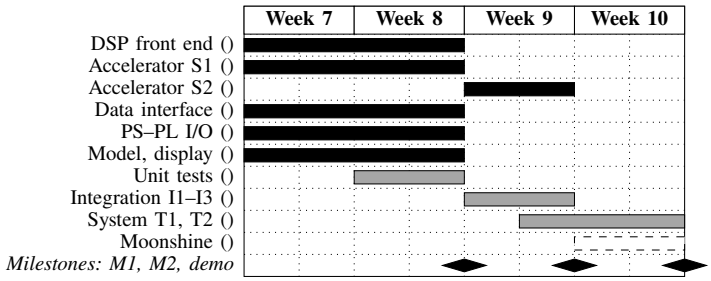


Fig. 6. Schedule (half weeks): black is build, grey is test, diamonds are M1, M2 and the demo; the reach bar runs only if T1 passes.

V. TASKS, TIMELINE AND RISKS

Task allocation. We split the design into five subsystems built in parallel, one per member (Table IX). They meet at two fixed formats, 64-band feature frames and QNC2 engine commands (opcode, tag, sizes), so each owner builds and unit-tests against recorded data before integration; backups own the neighbouring subsystem.

Timeline. We completed our initial tests of possible models in week 6. We benchmarked every candidate model on the board using only the CPU and all 4 threads, and Table II shows the results. All five subsystems are built and unit-tested in parallel in weeks 7–8, gated by M1 (Fig. 6). Integration tests I1–I3 run in week 9 alongside stage S2, and system test T1 at M2 decides whether the reach goal starts. Week 10 holds T2, the demo and the poster.

Risks and contingencies. Table X rates each risk by likelihood and impact (L/I); every fallback still leaves a working captioner and a measured answer.

TABLE X
RISKS AND CONTINGENCIES

Risk	L/I	Mitigation → fallback
Moving data cancels the PL's gain	H/H	S2 one command per chunk, weight cache, profiling → report the breakdown; CPU INT8
Chunking hurts latency or WER	M/H	Full-phrase re-decode for finals; tune hop and overlap → Moonshine on the same engine
Timing or URAM fails at 100 MHz	H/M	BRAM in place of URAM, smaller tiles → fewer MAC units
Int8 or the PL front end raises WER	M/H	Float log, acceptance rule → revert that layer or the FFT to the CPU
Microphone, clock or library faults	M/H	Start-up self-test, pinned versions → WAV replay for the demo
Reach: PS decoder limits Moonshine	H/L	Optional; profile it → report the finding
A member is away	L/H	Named backups (Table IX) → rebalance tasks

REFERENCES

[1] World Health Organization, "Hearing loss," Health topic. [Online]. Available: https://www.who.int/health-topics/hearing-loss (accessed Oct. 30, 2026).
[2] J. Nielsen, *Usability Engineering*. Academic Press, 1993.
[3] B. Thompson *et al.*, "Are caption metrics broken? Latency, deaf and hard of hearing user ratings, and bias across technologies," arXiv:2609.11408, 2026.
[4] *Kria K26 SOM Data Sheet (DS987)*, AMD.
[5] A. Radford *et al.*, "Robust speech recognition via large-scale weak supervision," arXiv:2212.04356, 2022.
[6] S. Majumdar *et al.*, "Citrinet: Closing the gap between non-autoregressive and autoregressive end-to-end models for automatic speech recognition," arXiv:2104.01721, 2021.
[7] M. Kudlur *et al.*, "Moonshine v2: Ergodic streaming encoder ASR for latency-critical speech applications," arXiv:2602.12241, 2026.
[8] S. Kriman *et al.*, "QuartzNet: Deep automatic speech recognition with 1D time-channel separable convolutions," in *Proc. IEEE ICASSP*, 2020.
[9] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "LibriSpeech: An ASR corpus based on public domain audio books," in *Proc. IEEE ICASSP*, 2015, pp. 5206–5210.
[10] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *Proc. IEEE ISSCC*, 2014.
[11] S. Kim *et al.*, "Integer-only zero-shot quantization for efficient speech recognition," in *Proc. IEEE ICASSP*, 2022.
[12] T. Stivers *et al.*, "Universals and cultural variation in turn-taking in conversation," *Proc. Natl. Acad. Sci. USA*, vol. 106, no. 26, pp. 10 587–10 592, 2009.
[13] *G.114: One-Way Transmission Time*, ITU-T, 2003.
[14] Artificial Analysis, "Speech to text methodology," 2026, accessed Oct. 2026. [Online]. Available: https://artificialanalysis.ai/methodology/speech-to-text
[15] C. Wang *et al.*, "VoxPopuli: A large-scale multilingual speech corpus for representation learning, semi-supervised learning and interpretation," in *Proc. ACL*, 2021.
[16] C. Munteanu, R. Baecker, G. Penn, E. Toms, and D. James, "The effect of speech recognition accuracy rates on the usefulness and usability of webcast archives," in *Proc. ACM CHI*, 2006, pp. 493–502.
[17] B. Funk, "AMD-Xilinx Kria KV260 starter kit: Exploring machine vision AI," *HotHardware*, Jun. 2022. [Online]. Available: https://hothardware.com/reviews/xilinx-kria-kv260-vision-ai-starter-kit-review